

Document number:	P0740R0
Date:	2017-07-14
Project:	C++ Extensions for Ranges, Library Working Group
Reply-to:	Casey Carter < Casey@Carter.net >

Ranges TS “Immediate” Issues from the July 2017 (Toronto) meeting

61: Review stated complexities of algorithms wrt the use of projections

Many stated complexities involving number of projection invocations are wrong by a factor of two. See [ericniebler/range-v3#148](http://ericniebler.com/range-v3/#148) and `min_element`, for example.

Proposed Resolution

Change `[alg.transform]/5` as follows:

```

4 Returns: {first1 + N, result + N} or make_tagged_tuple<tag::in1, tag::in2, [...]
5 Complexity: Exactly N applications of op or binary_op.
+5 Complexity: Exactly N applications of op or binary_op and the corresponding projection(s).
6 Remarks: result may be equal to first1 in case of unary transform, or to first1 [...]
```

Change `[sort]/2` as follows:

```

1 Effects: Sorts the elements in the range [first,last).
2 Complexity: O(N log(N)) (where N == last - first) comparisons.
+2 Complexity: O(N log(N)) (where N == last - first) comparisons, and twice as many applications
+ of the projection.
```

Change `[stable.sort]/2` as follows:

```

1 Effects: Sorts the elements in the range [first,last).
2 Complexity: It does at most N log2(N) (where N == last - first) comparisons; if enough extra
memory is available, it is N log(N).
+2 Complexity: Let N == last - first. If enough extra memory is available, N log(N) comparisons.
+ Otherwise, at most N log2(N) comparisons. In either case, twice as many applications of the
+ projection as the number of comparisons.
3 Remarks: Stable (ISO/IEC 14882:2014 §17.6.5.7).
```

Change `[partial.sort]/2` as follows:

```

1 Effects: Places the first middle - first sorted elements from the range [first,last) [...]
2 Complexity: It takes approximately (last - first) * log(middle - first) comparisons.
+2 Complexity: It takes approximately (last - first) * log(middle - first) comparisons, and exactly
+ as many applications of the projection.
```

Change `[partial.sort.copy]/3` as follows:

```

2 Returns: The smaller of: result_last or result_first + (last - first).
3 Complexity: Approximately (last - first) * log(min(last - first, result_last - result_first))
```

```

comparisons,
+3 Complexity: Approximately (last - first) * log(min(last - first, result_last - result_first))
+ comparisons, and exactly twice as many applications of the projection.

```

Change [sort.heap]/4 as follows:

```

3 Returns: last
4 Complexity: At most N log(N) comparisons (where N == last - first).
+4 Complexity: At most N log(N) comparisons (where N == last - first), and exactly twice as many
+ applications of the projection.

```

70: Why do neither reference types nor array types satisfy Destructible?

I must be missing something: why do we want neither reference types nor array types to satisfy `Destructible<>()`? Surely variables of such types can be destroyed.

I understand that `Destructible` is considered “the base of the hierarchy of object concepts” — but if it’s really needed, why isn’t there a `SingleObject<>()` concept for this purpose? Or, better yet, just thusly rename the current `Destructible`?

Proposed Resolution

Adopt [P0547R2: “Ranges TS: Assorted Object Concept Fixes”](#).

154: Raw pointer does not satisfy the requirements of `RandomAccessIterator`

In the requirement of `RandomAccessIterator<I>` requires that `I` must satisfy `TotallyOrderable<I>`. From the semantical requirement of 19.3.4 [concepts.lib.compare.totallyordered] p1, this requires that for all objects `a`, `b`, `c` of iterator type one of following is true: `a < b`, `b < a`, `b == a`. Non normative note placed below there requirements, states that this is not required to be true for not well-formed object (default constructed pointers/iterators), but does not exclude compare pointer to different arrays (which are well-formed).

Proposed Resolution

Modify the synopsis of the header `<experimental/ranges/functional>` in [function.objects] as follows:

```

// 8.3.2, comparisons:
template <class T = void>
requires EqualityComparable<T> || Same<T, void>
+ requires see below
struct equal_to;

template <class T = void>
requires EqualityComparable<T> || Same<T, void>
+ requires see below
struct not_equal_to;

template <class T = void>
requires StrictTotallyOrdered<T> || Same<T, void>
+ requires see below
struct greater;

template <class T = void>
requires StrictTotallyOrdered<T> || Same<T, void>
+ requires see below

```

```

struct less;

template <class T = void>
requires StrictTotallyOrdered<T> || Same<T, void>
+ requires see below
struct greater_equal;

template <class T = void>
requires StrictTotallyOrdered<T> || Same<T, void>
+ requires see below
struct less_equal;

template <> struct equal_to<void>;
template <> struct not_equal_to<void>;
[...]
```

Also modify the detailed specifications of the comparison function objects in [comparisons]:

1 The library provides basic function object classes for all of the comparison operators in the language (ISO/IEC 14882:2014 §5.9, ISO/IEC 14882:2014 §5.10).

```
+? In this section, BUILTIN_PTR_CMP(T, OP, U) for types T and U and where OP is
+ an equality (\cxxref{expr.eq}) or relational operator (\cxxref{expr.rel}) is
+ a boolean constant expression. BUILTIN_PTR_CMP(T, OP, U) is true if and only
+ if OP in the expression declval<T>() OP declval<U>() resolves to a built-in
+ operator comparing pointers.
```

```
+
+? There is an implementation-defined strict total ordering over all pointer
+ values of a given type. This total ordering is consistent with the partial
+ order imposed by the builtin operators <, >, <=, and >=.
```

```

template <class T = void>
  requires EqualityComparable<T> || Same<T, void>
+   || BUILTIN_PTR_CMP(const T&, ==, const T&)
struct equal_to {
  constexpr bool operator()(const T& x, const T& y) const;
};
```

```
-2 operator() returns x == y.
+2 operator() has effects equivalent to: return equal_to<>{}(x, y);
```

```

template <class T = void>
  requires EqualityComparable<T> || Same<T, void>
+   || BUILTIN_PTR_CMP(const T&, ==, const T&)
struct not_equal_to {
  constexpr bool operator()(const T& x, const T& y) const;
};
```

```
-3 operator() returns x != y.
+3 operator() has effects equivalent to: return !equal_to<>{}(x, y);
```

```

template <class T = void>
  requires StrictTotallyOrdered<T> || Same<T, void>
+   || BUILTIN_PTR_CMP(const T&, <, const T&)
struct greater {
  constexpr bool operator()(const T& x, const T& y) const;
};
```

```
-4 operator() returns x > y.
+4 operator() has effects equivalent to: return less<>{}(y, x);
```

```

template <class T = void>
  requires StrictTotallyOrdered<T> || Same<T, void>
+   || BUILTIN_PTR_CMP(const T&, <, const T&)
struct less {
  constexpr bool operator()(const T& x, const T& y) const;
};
```

~~5 operator() returns x < y.~~
+5 operator() has effects equivalent to return less<>{}(x, y);

```
template <class T = void>
    requires StrictTotallyOrdered<T> || Same<T, void>
+    || BUILTIN_PTR_CMP(const T&, <, const T&)
    struct greater_equal {
        constexpr bool operator()(const T& x, const T& y) const;
    };
};
```

~~6 operator() returns x >= y.~~
+6 operator() has effects equivalent to return !less<>{}(x, y);.

```
template <class T = void>
    requires StrictTotallyOrdered<T> || Same<T, void>
+    || BUILTIN_PTR_CMP(const T&, <, const T&)
    struct less_equal {
        constexpr bool operator()(const T& x, const T& y) const;
    };
};
```

~~7 operator() returns x <= y.~~
+7 operator() has effects equivalent to: return !less<>{}(y, x);

```
template <> struct equal_to<void> {
    template <class T, class U>
        requires EqualityComparableWith<T, U>
        constexpr auto operator()(T&& t, U&& u) const
        > decltype(std::forward<T>(t) == std::forward<U>(u));
+    requires EqualityComparableWith<T, U> || BUILTIN_PTR_CMP(T, ==, U)
+    constexpr bool operator()(T&& t, U&& u) const;
```

```
    typedef unspecified is_transparent;
};
```

~~8 operator() returns std::forward<T>(t) == std::forward<U>(u).~~
+8 Requires: If the expression std::forward<T>(t) == std::forward<U>(u) results in a call to a
+ built-in operator == comparing pointers of type P, the conversion sequences from both
+ T and U to P shall be equality-preserving (\ref{concepts.lib.general.equality}).

+?- Effects:

+ (?1) - If the expression std::forward<T>(t) == std::forward<U>(u) results in a call to a
+ built-in operator == comparing pointers of type P: returns false if either (the
+ converted value of) t precedes u or u precedes t in the implementation-defined
+ strict total order over pointers of type P and otherwise true.
+ (?2) - Otherwise, equivalent to: return std::forward<T>(t) == std::forward<U>(u);

```
template <> struct not_equal_to<void> {
    template <class T, class U>
        requires EqualityComparableWith<T, U>
        constexpr auto operator()(T&& t, U&& u) const
        > decltype(std::forward<T>(t) != std::forward<U>(u));
+    requires EqualityComparableWith<T, U> || BUILTIN_PTR_CMP(T, ==, U)
+    constexpr bool operator()(T&& t, U&& u) const;
```

```
    typedef unspecified is_transparent;
};
```

~~9 operator() returns std::forward<T>(t) != std::forward<U>(u).~~
+9 operator() has effects equivalent to:
+ return !equal_to<>{}(std::forward<T>(t), std::forward<U>(u));

```
template <> struct greater<void> {
    template <class T, class U>
        requires StrictTotallyOrderedWith<T, U>
        constexpr auto operator()(T&& t, U&& u) const
        > decltype(std::forward<T>(t) > std::forward<U>(u));
+    requires StrictTotallyOrderedWith<T, U> || BUILTIN_PTR_CMP(U, <, T)
+    constexpr bool operator()(T&& t, U&& u) const;
```

```

typedef unspecified is_transparent;
};

10 operator() returns std::forward<T>(t) > std::forward<U>(u).
+10 operator() has effects equivalent to:
+   return less<>{}(std::forward<U>(u), std::forward<T>(t));

template <> struct less<void> {
    template <class T, class U>
    requires StrictTotallyOrderedWith<T, U>
    constexpr auto operator()(T&& t, U&& u) const
    > decltype(std::forward<T>(t) < std::forward<U>(u));
+   requires StrictTotallyOrderedWith<T, U> || BUILTIN_PTR_CMP(T, <, U)
+   constexpr bool operator()(T&& t, U&& u) const;

    typedef unspecified is_transparent;
};

11 operator() returns std::forward<T>(t) < std::forward<U>(u).
+11 Requires: If the expression std::forward<T>(t) < std::forward<U>(u) results in a call to a
+   built-in operator < comparing pointers of type P, the conversion sequences from both
+   T and U to P shall be equality-preserving (\ref{concepts.lib.general.equality}). For any
+   expressions ET and EU such that decltype((ET)) is T and decltype((EU)) is U, exactly one
+   of less<>{}(ET, EU), less<>{}(EU, ET) or equal_to<>{}(ET, EU) shall be true.

+?- Effects:
+ (?..1) - If the expression std::forward<T>(t) < std::forward<U>(u) results in a call to a
+   built-in
+   operator < comparing pointers of type P: returns true if (the converted value of) t
+   precedes u in the implementation-defined strict total order over pointers of type P
+   and otherwise false.
+ (?..2) - Otherwise, equivalent to: return std::forward<T>(t) < std::forward<U>(u);

template <> struct greater_equal<void> {
    template <class T, class U>
    requires StrictTotallyOrderedWith<T, U>
    constexpr auto operator()(T&& t, U&& u) const
    > decltype(std::forward<T>(t) >= std::forward<U>(u));
+   requires StrictTotallyOrderedWith<T, U> || BUILTIN_PTR_CMP(T, <, U)
+   constexpr bool operator()(T&& t, U&& u) const;

    typedef unspecified is_transparent;
};

12 operator() returns std::forward<T>(t) >= std::forward<U>(u).
+12 operator() has effects equivalent to:
+   return !less<>{}(std::forward<T>(t), std::forward<U>(u));

template <> struct less_equal<void> {
    template <class T, class U>
    requires StrictTotallyOrderedWith<T, U>
    constexpr auto operator()(T&& t, U&& u) const
    > decltype(std::forward<T>(t) <= std::forward<U>(u));
+   requires StrictTotallyOrderedWith<T, U> || BUILTIN_PTR_CMP(U, <, T)
+   constexpr bool operator()(T&& t, U&& u) const;

    typedef unspecified is_transparent;
};

13 operator() returns std::forward<T>(t) <= std::forward<U>(u).
+13 operator() has effects equivalent to:
+   return !less<>{}(std::forward<U>(u), std::forward<T>(t));

14 For templates greater, less, greater_equal, and less_equal, the specializations for any
    pointer type yield a total order, even if the built in operators <, >, <=, >= do not.

```

156: Validity of references obtained from out-of-lifetime iterators

Summary

By removing the requirement in C++14 [forward.iterators]/6:

If a and b are both dereferenceable, then $a == b$ if and only if $*a$ and $*b$ are bound to the same object.

the Ranges TS inadvertently allows “stashing” forward iterators, which notoriously break `reverse_iterator` and e.g. `return *(some_local_iterator + 4)`.

Detailed discussion

Iterators that return a reference to a member object are known as “stashing” iterators - `istream_iterator` is the classic example. They famously do not work with `reverse_iterator`, since they violate [forward.iterators]/6:

If a and b are both dereferenceable, then $a == b$ if and only if $*a$ and $*b$ are bound to the same object.

[“various concept tweaks” committed June 1](#) changed this requirement in the TS to:

If `\tcode{a}` and `\tcode{b}` are both dereferenceable, then `\tcode{a == b}` if and only if `\tcode{*a}` is equal to `\tcode{*b}`.

So that the requirement has well-defined meaning when $*a$ and $*b$ are not objects.

[“Language for equality-preserving-by-default expressions” committed July 3](#) removed the requirement completely, since

- $*a == *b$ if $a == b$ is implied by the equality preservation of $*$ from `Readable`, and
- $a == b$ if $*a == *b$ requires that a value appears at most once in any given range, which was not intended.

I believe the TS wording now allows “stashing” iterators to satisfy `Forward`, despite that many reasonable operations simply don’t work with them. Iterators *denote* elements, they must not *own* them: the expectation is that the lifetime of denoted object elements is the lifetime of the range, *not* the lifetime of the iterator. We need to bring back a requirement that forbids “stashing” `Forward` iterators. This should cause no hardship to TS users since `Forward` iterators can have prvalue types.

Proxy reference types make it challenging to develop a simple requirement; the straightforward “If `reference_t<I>` is a reference type, then $&*a == &*b$ iff $a == b$ ” doesn’t work. The high-level wording “If iterator values denote objects, then $a == b$ iff a and b denote the same object.” seems clearer to me, but I’m accustomed to thinking of iterators as either denoting objects or denoting values. I’m not sure someone coming straight to the TS from C++14 would understand. I also don’t think the “If..dereferenceable” qualifier since (a) past-the-end iterators denote the same past-the-end object, and (b) singular iterators won’t be in the domain of $==$ and can therefore do anything they like.

Proposed Resolution

Add a new paragraph to section “Concept `ForwardIterator`” ([`iterators.forward`]) after paragraph 4:

1. Pointers and references obtained from a forward iterator into a range $[i, s)$ must remain valid while $[i, s)$ continues to denote a range.

167: `ConvertibleTo` should require both implicit and explicit conversion

Currently, `ConvertibleTo` checks only for *implicit* convertibility (a-la `is_convertible`). It would be highly surprising for generic code for an *explicit* conversion to fail, or to succeed but yield a different result.

Proposed Resolution

(Also fixes [#314](#).)

Replace the contents of `[concepts.lib.corelang.convertibleto]` with:

```
template <class From, class To>
concept bool ConvertibleTo() {
    return is_convertible<From, To>::value &&
        requires (From (&f)()) {
            static_cast<To>(f());
        };
}
```

1 Let test be the invented function:

```
To test(From (&f)()) {
    return f();
}
```

and let `f` be a function with no arguments and return type `From` such that `f()` is equality preserving. Then `ConvertibleTo<From, To>()` is satisfied if and only if:

(1.1) - `To` is not an object or reference-to-object type, or `static_cast<To>(f())` is equal to `test(f)`.

(1.2) - `From` is not a reference-to-object type, or

(1.2.1) - If `From` is an rvalue reference to a non const-qualified type, the resulting state of the object referenced by `f()` after either above expression is valid but unspecified (`[lib.types.movedfrom]`).

(1.2.2) - Otherwise, the object referred to by `f()` is not modified by either above expression.

2 There need not be any subsumption relationship between `ConvertibleTo<From, To>()` and `is_convertible<From, To>::value`.

[170](#): `unique_copy` and LWG 2439

The declaration of `unique_copy` is underconstrained when the iterator category of the source range is not forward. It should probably be brought into line with the resolution of [LWG2439](#).

Proposed Resolution

Update the declarations of `unique_copy` in the synopsis of `<experimental/ranges/algorithm>` in `[algorithms.general]` as follows:

```
template <InputIterator I, Sentinel<I> S, WeaklyIncrementable O,
        class Proj = identity, IndirectRelation<projected<I, Proj>> R = equal_to<>>
    requires IndirectlyCopyable<I, O>() && (ForwardIterator<I>() ||
        ForwardIterator<O>() || IndirectlyCopyableStorable<I, O>())
+   requires IndirectlyCopyable<I, O>() &&
+   (ForwardIterator<I>() ||
+   (InputIterator<O>() && Same<value_type_t<I>, value_type_t<O>>()) ||
+   IndirectlyCopyableStorable<iterator_t<Rng>, O>())
    tagged_pair<tag::in(I), tag::out(O)>
    unique_copy(I first, S last, O result, R comp = R{}, Proj proj = Proj{});
```

```

template <InputRange Rng, WeaklyIncrementable O, class Proj = identity,
    IndirectRelation<projected<iterator_t<Rng>, Proj>> R = equal_to<>>
    requires IndirectlyCopyable<iterator_t<Rng>, O>() &&
    (ForwardIterator<iterator_t<Rng>>() || ForwardIterator<O>() ||
+    (ForwardIterator<iterator_t<Rng>>() ||
+    (InputIterator<O>() && Same<value_type_t<I>, value_type_t<O>>()) ||
    IndirectlyCopyableStorable<iterator_t<Rng>, O>())
    tagged_pair<tag::in(safe_iterator_t<Rng>), tag::out(O)>
    unique_copy(Rng&& rng, O result, R comp = R{}, Proj proj = Proj{});

```

174: Swappable concept and P0185 swappable traits

...have inconsistent interfaces. Our Swappable<T, U> is roughly equivalent to C++17's is_swappable_with<T, U>, so is_swappable<T> - which is equivalent to is_swappable_with<T&, T&> - is roughly equivalent to Swappable<T&, T&>. There's enormous potential for confusion in this disparity.

We need to bring the concept definitions to parity with the WP traits by renaming Swappable<T, U> to SwappableWith, and defining Swappable<T> to be roughly equivalent to SwappableWith<T&, T&>:

Proposed Resolution

(Includes the proposed resolution of [#379](#).)

Change the definition of concept Swappable ([concepts.lib.corelang.swappable]) to:

```

template <class T>
concept bool Swappable =
    requires(T& a, T& b) {
        ranges::swap(a, b);
    };

template <class T, class U>
concept bool SwappableWith =
    CommonReference<
        const remove_reference_t<T>&,
        const remove_reference_t<U>&> &&
    requires(T&& t, U&& u) {
        ranges::swap(std::forward<T>(t), std::forward<T>(t));
        ranges::swap(std::forward<U>(u), std::forward<U>(u));
        ranges::swap(std::forward<T>(t), std::forward<U>(u));
        ranges::swap(std::forward<U>(u), std::forward<T>(t));
    };

```

Change the definition of Movable ([concepts.lib.object.movable]) as follows (includes changes from [P0547R1](#)):

```

template <class T>
concept bool Movable =
    std::is_object<T>::value && // see below
    MoveConstructible<T> &&
    Assignable<T&, T> &&
    Swappable<T&>;
+    Swappable<T>;

```

In the class synopsis of class template tagged ([taggedup.tagged]/p2), make the following changes:

```

tagged& operator=(U&& u) noexcept(see below );
void swap(tagged& that) noexcept(see below )
    requires Swappable<Base&>;

```



```
+ requires Swappable<Base>;
    friend void swap(tagged&, tagged&) noexcept(see below )
requires Swappable<Base&>;
+ requires Swappable<Base>;
};
```

Make the same changes to the detailed specifications of `tagged::swap` and the non-member `swap(tagged&, tagged&)` overload in [taggedup.tagged]/p20 and p23

176: Relax requirements on `replace` and `replace_if`

`replace` and `replace_if` are specified to require a forward range. This is necessary to support the effects statement: “Substitutes elements referred by the iterator `i` in the range `[first, last)` with `new_value`”, since weaker iterators might not reference elements.

I think the requirement can be relaxed to input if we rephrase the effects as “Assigns `new_value` through each iterator `i` in the range `[first, last)` that satisfies the condition ...”. (Note that we already separately require `Writable<I, const T&>()`.)

Proposed Resolution

In the synopsis of <experimental/ranges/algorithm> ([algorithms.general]/p2), make the following changes:

```
template <ForwardIterator I, Sentinel<I> S, class T1, class T2, class Proj = identity>
+template <InputIterator I, Sentinel<I> S, class T1, class T2, class Proj = identity>
    requires Writable<I, const T2&>() &&
        IndirectRelation<equal_to<>, projected<I, Proj>, const T1*>()
I
    replace(I first, S last, const T1& old_value, const T2& new_value, Proj proj = Proj{});

template <ForwardRange Rng, class T1, class T2, class Proj = identity>
+template <InputRange Rng, class T1, class T2, class Proj = identity>
    requires Writable<iterator_t<Rng>, const T2&>() &&
        IndirectRelation<equal_to<>, projected<iterator_t<Rng>, Proj>, const T1*>()
    safe_iterator_t<Rng>
    replace(Rng&& rng, const T1& old_value, const T2& new_value, Proj proj = Proj{});

template <ForwardIterator I, Sentinel<I> S, class T, class Proj = identity>
+template <InputIterator I, Sentinel<I> S, class T, class Proj = identity>
    IndirectPredicate<projected<I, Proj>> Pred>
    requires Writable<I, const T&>()
I
    replace_if(I first, S last, Pred pred, const T& new_value, Proj proj = Proj{});

template <ForwardRange Rng, class T, class Proj = identity>
+template <InputRange Rng, class T, class Proj = identity>
    IndirectPredicate<projected<iterator_t<Rng>, Proj>> Pred>
    requires Writable<iterator_t<Rng>, const T&>()
    safe_iterator_t<Rng>
    replace_if(Rng&& rng, Pred pred, const T& new_value, Proj proj = Proj{});
```

In [alg.replace], change the signatures of `replace` and `replace_if` to match the ones above. Also, make the following change to p1:

```
1. Effects: Substitutes elements referred by the iterator i in the range [first, last) with new_value
+1. Effects: Assigns new_value through each iterator i in the range [first, last)
    when the following corresponding conditions hold:
    invoke(proj, *i) == old_value, invoke(pred, invoke(proj, *i)) != false.
```

211: Add new header <experimental/range/range>

Where do `enable_view`, `disable_sized_range`, and `view_base` live? Or the Range concepts? Do we need a <experimental/ranges/range> header?

Heck, where do the Iterator concepts live? They don't appear in the iterator header synopsis.

Solution description

The problems with the concepts and iterator synopses have been handled as editorial changes in 1862bd8 and e15540b. This resolution is describes as a diff to those changes. It suggests adding a new header to hold all range-related functionality: <experimental/ranges/range>. This gives us a place to put the range customization points (begin, end, size, etc.), the range concepts, and range utilities.

It will also give us a place to put future range-related functionality, such as a view facade and adaptors, without further bloating <experimental/ranges/iterator>, which is quite heavy already.

The following resolution has been implemented in [#328](#), but not merged pending LWG review.

Proposed Resolution

To “Table 1 - Ranges TS library headers”, add <experimental/ranges/range>.

To “Table 2 - Library categories”, add a row for “Ranges library” between the “Iterators library” and the “Algorithms library”.

After [library.general]/p6, add a new paragraph that reads:

The ranges library ([\ref{ranges}](#)) describes components for dealing with ranges of elements.

Change [iterators.general]/1 as follows:

This Clause describes components that \Cpp programs may use to perform iterations over containers (Clause [\cxxref{containers}](#)), streams~([\cxxref{iostream.format}](#)), ~~stream buffers~([\cxxref{stream.buffers}](#)),~~ ~~and ranges~([\ref{ranges}](#)).~~ ~~+and stream buffers~([\cxxref{stream.buffers}](#)).~~

From table “Table 5 — Iterators library summary” [tab:iterators.lib.summary], delete the last line, which reads “Ranges”.

From [iterator.synopsis], delete the following lines:

```
// \ref{iterator.range}, range access:  
namespace {  
constexpr unspecified begin = unspecified;  
constexpr unspecified end = unspecified;  
constexpr unspecified cbegin = unspecified;  
constexpr unspecified cend = unspecified;  
constexpr unspecified rbegin = unspecified;  
constexpr unspecified rend = unspecified;  
constexpr unspecified crbegin = unspecified;  
constexpr unspecified crend = unspecified;  
}
```

```

// \ref{range.primitives}, range primitives:
namespace {
constexpr unspecified size = unspecified;
constexpr unspecified empty = unspecified;
constexpr unspecified data = unspecified;
constexpr unspecified cdata = unspecified;
}
template <Range R>
difference_type_t<iterator_t<R>> distance(R&& r);
template <SizedRange R>
difference_type_t<iterator_t<R>> distance(R&& r);

```

Between Clause 6 (Iterators) and Clause 7 (Algorithms), add a new Clause “Ranges” with stable name [ranges]. Move [ranges.general] from [iterators] into the new [ranges] clause, with the following changes:

```

1 This subclause describes components for dealing with
+1 This Clause describes components for dealing with
ranges of elements.
2 The following subclauses describe range and view
requirements, and components for range primitives,
predefined ranges, and stream ranges, as summarized in
+as summarized in
Table 7.

```

In Table 7 - Ranges library summary, move “Requirements” from the first line to the last and change the header from <experimental/ranges/iterator> to <experimental/ranges/range>.

Add a subclause “Header <experimental/ranges/range> synopsis” with stable name [range.synopsis] with the following content:

```

#include <experimental/ranges/iterator>

namespace std { namespace experimental { namespace ranges { inline namespace v1 {
    // \ref{range.access}, range access:
    namespace {
        constexpr unspecified begin = unspecified;
        constexpr unspecified end = unspecified;
        constexpr unspecified cbegin = unspecified;
        constexpr unspecified cend = unspecified;
        constexpr unspecified rbegin = unspecified;
        constexpr unspecified rend = unspecified;
        constexpr unspecified crbegin = unspecified;
        constexpr unspecified crend = unspecified;
    }

    // \ref{range.primitives}, range primitives:
    namespace {
        constexpr unspecified size = unspecified;
        constexpr unspecified empty = unspecified;
        constexpr unspecified data = unspecified;
        constexpr unspecified cdata = unspecified;
    }

    template <class T>
    using iterator_t = decltype(ranges::begin(declval<T>()));

    template <class T>
    using sentinel_t = decltype(ranges::end(declval<T>()));

    template <class>
    constexpr bool disable_sized_range = false;

    template <class T>
    struct enable_view { };

```

```

struct view_base { };

// \ref{ranges.requirements}, range requirements:

// \ref{ranges.range}, Range:
template <class T>
concept bool Range() {
    return see below;
}

// \ref{ranges.sized}, SizedRange:
template <class T>
concept bool SizedRange() {
    return see below;
}

// \ref{ranges.view}, View:
template <class T>
concept bool View() {
    return see below;
}

// \ref{ranges.bounded}, BoundedRange:
template <class T>
concept bool BoundedRange() {
    return see below;
}

// \ref{ranges.input}, InputRange:
template <class T>
concept bool InputRange() {
    return see below;
}

// \ref{ranges.output}, OutputRange:
template <class R, class T>
concept bool OutputRange() {
    return see below;
}

// \ref{ranges.forward}, ForwardRange:
template <class T>
concept bool ForwardRange() {
    return see below;
}

// \ref{ranges.bidirectional}, BidirectionalRange:
template <class T>
concept bool BidirectionalRange() {
    return see below;
}

// \ref{ranges.random.access}, RandomAccessRange:
template <class T>
concept bool RandomAccessRange() {
    return see below;
}

// \ref{range.utilities}, range utilities:
template <Range R>
difference_type_t<iterator_t<R>> distance(R&& r);

template <SizedRange R>
difference_type_t<iterator_t<R>> distance(R&& r);
}}}}

```

Move [iterator.range] from [iterators] into the new [ranges] clause right after the synopsis. Change its stable name to [range.access] (and fix all references).

Add a new paragraph to [range.access]:

1 In addition to being available via inclusion of the <experimental/ranges/range> header, the customization point objects in \ref{range.access} are available when <experimental/ranges/iterator> is included.

Change all stable names within [range.access] from [iterator.range.*] to [range.access.*] (e.g. [iterator.range.begin] becomes [range.access.begin]). Fix up all references.

Move [range.primitives] from [iterators] to the [ranges] clause right after [ranges.access].

Add a new paragraph to [range.primitives]:

1 In addition to being available via inclusion of the <experimental/ranges/range> header, the customization point objects in \ref{range.primitives} are available when <experimental/ranges/iterator> is included.

Move section “Range requirements” [ranges.requirements] from [iterator] to the [ranges] clause, right after [range.primitives]. Promote it to a subclause; likewise, promote all its (sub)sections up one level.

From [ranges.range]/p1, remove the definitions of the iterator_t and sentinel_t template aliases. (They now live in the <experimental/ranges/range> synopsis.)

From [ranges.sized]/p1, remove the definition of disable_sized_range.

From [ranges.view]/p2, remove the definitions of enable_view and view_base.

Create a new subclause “Range utilities” ([range.utilities]) after “Range requirements” ([ranges.requirements]). Move the specification of distance from [range.primitives] into a new section “distance” ([range.distance]).

229: Assignable concept looks wrong

For Assignable we have:

```
template <class T, class U>
concept bool Assignable() {
    return Common<T, U>() && requires(T&& t, U&& u) {
        { std::forward<T>(t) = std::forward<U>(u) } -> Same<T&>;
    };
}
```

1. Let t be an lvalue of type T ,

The “*Let t be an lvalue of type T* ” is at odds with the concept definition. It needs the lvalue/rvalue dance.

Also, the application of `==` to entities whose types aren’t constrained to satisfy `EqualityComparable` is meaningless; `uu == v` and `t == uu` should use “is equal to.”

Proposed Resolution:

Adopt [P0547R2: “Ranges TS: Assorted Object Concept Fixes”](#).

250: Do common_iterator's copy/move ctors/operators need to be specified?

The members of common_iterator are worded to permit a union-like implementation. I think we need to say what copying/assigning these things do. Also: what do we say about constexpr/noexcept?

Proposed Resolution

(Wording relative to N4671. This wording also resolves [#436](#) “common_iterator’s destructor should not be specified in [common.iter.op=]”).

Strike the destructor declaration from the synopsis of class common_iterator in [common.iterator] as follows:

```
[...]
common_iterator& operator=(const common_iterator<ConvertibleTo<I>, ConvertibleTo<S>>& u);
common_iterator();
see below operator*();
[...]
```

Strike [common.iterator]/1 that begins “Note: It is unspecified whether common_iterator’s members iter and sentinel ...”

Change [common.iter.op.const] as follows:

```
common_iterator();

1 Effects: Constructs a common_iterator, value-initializing is_sentinel
+ , sentinel,
and iter. Iterator operations applied to the resulting iterator have defined behavior if and
only if the corresponding operations are defined on a value-initialized iterator of type I.
2 Remarks: It is unspecified whether any initialization is performed for sentinel.

common_iterator(I i);

3 Effects: Constructs a common_iterator, initializing is_sentinel with false
and
+ ,
iter with i
+ , and value-initializing sentinel
.

4 Remarks: It is unspecified whether any initialization is performed for sentinel.

common_iterator(S s);

5 Effects: Constructs a common_iterator, initializing is_sentinel with true and sentinel with s
+ , and value-initializing iter
.

6 Remarks: It is unspecified whether any initialization is performed for iter.

common_iterator(const common_iterator<ConvertibleTo<I>, ConvertibleTo<S>>& u);

7 Effects: Constructs a common_iterator, initializing is_sentinel with u.is_sentinel
+ , iter with u.iter, and sentinel with u.sentinel
.

(7.1) If u.is_sentinel is true, sentinel is initialized with u.sentinel.
(7.2) If u.is_sentinel is false, iter is initialized with u.iter.
8 Remarks:
```

~~(8.1) If u.is_sentinel is true, it is unspecified whether any initialization is performed for iter.~~
~~(8.2) If u.is_sentinel is false, it is unspecified whether any initialization is performed for sentinel.~~

Change [common.iter.op=] as follows:

```
common_iterator& operator=(const common_iterator<ConvertibleTo<I>, ConvertibleTo<S>>& u);

1 Effects: Assigns u.is_sentinel to is_sentinel
+ , u.iter to iter, and u.sentinel to sentinel
.

(1.1) If u.is_sentinel is true, assigns u.sentinel to sentinel.
(1.2) If u.is_sentinel is false, assigns u.iter to iter.
Remarks:
(1.3) If u.is_sentinel is true, it is unspecified whether any operation is performed on iter.
(1.4) If u.is_sentinel is false, it is unspecified whether any operation is performed on sentinel.

2 Returns: *this

~common_iterator();
3 Effects: Destroys all members that are currently initialized.
```

255: DerivedFrom should be “publicly and unambiguously”

Proposed Resolution

Change [concepts.lib.corelang.derived] as follows:

```
template <class T, class U>
concept bool DerivedFrom() {
return see below ;
+ return is_base_of<U, T>::value &&
+ is_convertible<remove_cv_t<T>*, remove_cv_t<U>*>::value; // see below
}

1 DerivedFrom<T, U>() is satisfied if and only if is_base_of<U, T>::value is true.
+1 There need not be a subsumption relationship between DerivedFrom<T, U>() and either
+ is_base_of<U, T>::value or is_convertible<remove_cv_t<T>*, remove_cv_t<U>*>::value.
+
+2 [Note: DerivedFrom<T, U>() is satisfied if and only if T is publicly and unambiguously
+ derived from U, or T and U are the same class type ignoring cv qualifiers.-end note]
```

256: Add constexpr to advance, distance, next, and prev

Proposed Resolution

Adopt [P0579R1](#): “constexpr for <experimental/ranges/iterator>”.

261: Restrict alg.general changes from P0370 to apply only to the range-and-a-half algorithms

..as directed during LWG review of P0370. LWG is uncomfortable with the uncertainty this wording introduces, and would prefer to “limit the scope of the potential damage” to only the deprecated range-and-a-half algorithms.

Proposed resolution

(This wording assumes the PRs of [#286](#) and [#379](#) have been applied)

Strike para [algorithms.general]/10 which begins: “Some algorithms declare both an overload that takes a Range and an Iterator, and an overload that takes two Range parameters.”

Strike para [algorithms.general]/12 which begins, “Despite that the algorithm declarations nominally accept parameters by value [...]”

Replace the entire content of Annex A.2 [depr.algo.range-and-a-half] with:

1 The following algorithm signatures are deemed unsafe and are deprecated in this document.

2 Overloads of algorithms that take a Range argument and a forwarding reference parameter `first2_` behave as if they are implemented by calling `begin` and `end` on the Range and dispatching to the overload that takes separate iterator and sentinel arguments, perfectly forwarding `first2_`.

```
template <InputIterator I1, Sentinel<I1> S1, class I2, class Pred = equal_to<>,
         class Proj1 = identity, class Proj2 = identity>
requires InputIterator<decay_t<I2>> && !Range<I2> &&
IndirectPredicate<Pred, projected<I1, Proj1>, projected<decay_t<I2>, Proj2>>
tagged_pair<tag::in1(I1), tag::in2(decay_t<I2>>
    mismatch(I1 first1, S1 last1, I2&& first2_, Pred pred = Pred{}),
    Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{}));
```

```
template <InputRange Rng1, class I2, class Pred = equal_to<>,
         class Proj1 = identity, class Proj2 = identity>
requires InputIterator<decay_t<I2>> && !Range<I2> &&
IndirectPredicate<Pred, projected<iterator_t<Rng1>, Proj1>,
    projected<decay_t<I2>, Proj2>>
tagged_pair<tag::in1(safe_iterator_t<Rng1>), tag::in2(decay_t<I2>>
    mismatch(Rng1&& rng1, I2&& first2_, Pred pred = Pred{}),
    Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{}));
```

3 Effects: Equivalent to:

return mismatch(first1, last1, std::forward<I2>(first2_), unreachable{}, pred, proj1, proj2);, except that the underlying algorithm never increments `first2` more than `last1 - first1` times.

```
template <InputIterator I1, Sentinel<I1> S1, class I2,
         class Pred = equal_to<>, class Proj1 = identity, class Proj2 = identity>
requires InputIterator<decay_t<I2>> && !Range<I2> &&
IndirectlyComparable<I1, decay_t<I2>, Pred, Proj1, Proj2>
bool equal(I1 first1, S1 last1,
    I2&& first2_, Pred pred = Pred{}),
    Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{});
```

```
template <InputRange Rng1, class I2, class Pred = equal_to<>,
         class Proj1 = identity, class Proj2 = identity>
requires InputIterator<decay_t<I2>> && !Range<I2> &&
IndirectlyComparable<iterator_t<Rng1>, decay_t<I2>, Pred, Proj1, Proj2>
bool equal(Rng1&& rng1, I2&& first2_, Pred pred = Pred{}),
    Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{});
```


4 *Effects*: Equivalent to:

```
return first1 == mismatch(first1, last1, std::forward<I2>(first2_), pred, proj1, proj2).in1();
```

```
template <ForwardIterator I1, Sentinel<I1> S1, class I2,
    class Pred = equal_to<>, class Proj1 = identity, class Proj2 = identity>
requires ForwardIterator<decay_t<I2>> && !Range<I2> &&
    IndirectlyComparable<I1, decay_t<I2>, Pred, Proj1, Proj2>
bool is_permutation(I1 first1, S1 last1, I2&& first2_, Pred pred = Pred{},
    Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{});
```

```
template <ForwardRange Rng1, class I2, class Pred = equal_to<>,
    class Proj1 = identity, class Proj2 = identity>
requires ForwardIterator<decay_t<I2>> && !Range<I2> &&
    IndirectlyComparable<iterator_t<Rng1>, decay_t<I2>, Pred, Proj1, Proj2>
bool is_permutation(Rng1&& rng1, I2&& first2_, Pred pred = Pred{},
    Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{});
```

5 *Effects*: Equivalent to:

```
auto first2 = std::forward<I2>(first2_);
return is_permutation(first1, last1, first2, next(first2, distance(first1, last1)),
    pred, proj1, proj2);
```

```
template <ForwardIterator I1, Sentinel<I1> S1, class I2>
requires ForwardIterator<decay_t<I2>> && !Range<I2> &&
    IndirectlySwappable<I1, decay_t<I2>>
tagged_pair<tag::in1(I1), tag::in2(decay_t<I2>>)>
    swap_ranges(I1 first1, S1 last1, I2&& first2_);
```

```
template <ForwardRange Rng, class I>
requires ForwardIterator<decay_t<I>> && !Range<I> &&
    IndirectlySwappable<iterator_t<Rng>, decay_t<I>>
tagged_pair<tag::in1(safe_iterator_t<Rng>), tag::in2(decay_t<I>>)>
    swap_ranges(Rng&& rng1, I&& first2_);
```

6 *Effects*: Equivalent to:

```
auto first2 = std::forward<I2>(first2_);
return swap_ranges(first1, last1, first2, next(first2, distance(first1, last1)),
    pred, proj1, proj2);
```

```
template <InputIterator I1, Sentinel<I1> S1, class I2, WeaklyIncrementable O,
    class F, class Proj1 = identity, class Proj2 = identity>
requires InputIterator<decay_t<I2>> && !Range<I2> &&
    Writable<O, indirect_result_of_t<F&(projected<I1, Proj1>,
        projected<decay_t<I2>, Proj2>>>>
tagged_tuple<tag::in1(I1), tag::in2(decay_t<I2>), tag::out(O)>
    transform(I1 first1, S1 last1, I2&& first2_, O result,
        F binary_op, Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{});
```

```
template <InputRange Rng, class I, WeaklyIncrementable O, class F,
    class Proj1 = identity, class Proj2 = identity>
requires InputIterator<decay_t<I>> && !Range<I> &&
    Writable<O, indirect_result_of_t<F&(
        projected<iterator_t<Rng>, Proj1>, projected<decay_t<I>, Proj2>>>>
tagged_tuple<tag::in1(safe_iterator_t<Rng>), tag::in2(decay_t<I>), tag::out(O)>
    transform(Rng&& rng1, I&& first2_, O result,
        F binary_op, Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{});
```

7 *Effects*: Equivalent to:

```
return transform(first1, last1, std::forward<I2>(first2_), unreachable{}, pred, proj1, proj2);,
```

except that the underlying algorithm never increments `first2` more than `last1 - first1` times.

262: Use expression-equivalent in definitions of CPOs

...which is not part of the general library meaning of “Effects: Equivalent to.” We need to document this explicitly at each occurrence, or introduce blanket wording that covers the different usage in CPO descriptions, or introduce a new term (e.g. “expression-equivalent-to”).

[NOTE: C++14’s `DECAY_COPY` pseudo-macro is defined in terms of a `decay_copy` function which is *not* `constexpr` or `noexcept`, so nowhere can we say that something is *expression-equivalent to* an expression involving `DECAY_COPY` if we want to retain `constexpr`- and `noexcept`-ness ... unless we define our own `DECAY_COPY` that preserves `constexpr`- and `noexcept`-ness. — *end note*]

Proposed Resolution

After [iterator.requirements.general], add a new subsection “`decay_copy`” (stable name [iterator.decaycopy]), with the following contents:

`decay_copy` [iterator.decaycopy]

1 Several places in this Clause use the expression `DECAY_COPY(x)`, which is equivalent to `decay_t<decltype((x))>(x)`.

Change [iterator.range.begin]/p1 to read:

The name `begin` denotes a customization point object ([customization.point.object]).
The effect of the expression `ranges::begin(E)` for some expression `E` is
~~equivalent to:~~
+expression-equivalent to:

Change [iterator.range.end]/p1 to read:

The name `end` denotes a customization point object ([customization.point.object]).
The effect of the expression `ranges::end(E)` for some expression `E` is
~~equivalent to:~~
+expression-equivalent to:

Change [iterator.range.cbegin]/p1 to read:

The name `cbegin` denotes a customization point object ([customization.point.object]).
The effect of the expression `ranges::cbegin(E)` for some expression `E` of type `T` is
~~equivalent to `ranges::begin(static_const<const T&>(E))`.~~
+expression-equivalent to `ranges::begin(static_const<const T&>(E))`.

Change [iterator.range.cend]/p1 to read:

The name `cend` denotes a customization point object ([customization.point.object]).
The effect of the expression `ranges::cend(E)` for some expression `E` of type `T` is
~~equivalent to `ranges::end(static_const<const T&>(E))`.~~
+expression-equivalent to `ranges::end(static_const<const T&>(E))`.

Change [iterator.range.rbegin]/p1 to read:

The name `rbegin` denotes a customization point object ([customization.point.object]).
The effect of the expression `ranges::rbegin(E)` for some expression `E` is

~~equivalent to:~~
+expression-equivalent to:

Change [iterator.range.rend]/p1 to read:

The name `rend` denotes a customization point object ([customization.point.object]).
The effect of the expression `ranges::rend(E)` for some expression `E` is

~~equivalent to:~~
+expression-equivalent to:

Change [iterator.range.crbegin]/p1 to read:

The name `crbegin` denotes a customization point object ([customization.point.object]).
The effect of the expression `ranges::crbegin(E)` for some expression `E` of type `T` is

~~equivalent to `ranges::rbegin(static_cast<const T>(E))`.~~
+expression-equivalent to `ranges::rbegin(static_cast<const T>(E))`.

Change [iterator.range.crend]/p1 to read:

The name `crend` denotes a customization point object ([customization.point.object]).
The effect of the expression `ranges::crend(E)` for some expression `E` of type `T` is

~~equivalent to `ranges::rend(static_cast<const T>(E))`.~~
+expression-equivalent to `ranges::rend(static_cast<const T>(E))`.

Change [range.primitives.size]/p1 to read:

The name `size` denotes a customization point object ([customization.point.object]).
The effect of the expression `ranges::size(E)` for some expression `E` with type `T` is

~~equivalent to:~~
+expression-equivalent to:
~~(1.1) - `extent<T>::value` if `T` is an array type (ISO/IEC 14882:2014 §3.9.2).~~
+(1.1) - `DECAY_COPY(extent<T>::value)` if `T` is an array type (ISO/IEC 14882:2014 §3.9.2).

Change [range.primitives.empty]/p1 to read:

The name `empty` denotes a customization point object ([customization.point.object]).
The effect of the expression `ranges::empty(E)` for some expression `E` is

~~equivalent to:~~
+expression-equivalent to:

Change [range.primitives.data]/p1 to read:

The name `data` denotes a customization point object ([customization.point.object]).
The effect of the expression `ranges::data(E)` for some expression `E` is

~~equivalent to:~~
+expression-equivalent to:

Change [range.primitives.cdata]/p1 to read:

The name `cdata` denotes a customization point object ([customization.point.object]).
The effect of the expression `ranges::cdata(E)` for some expression `E` of type `T` is

~~equivalent to `ranges::data(static_cast<const T>(E))`.~~
+expression-equivalent to `ranges::data(static_cast<const T>(E))`.

284: `iter_move` and `iter_swap` need to say when they are `noexcept` and `constexpr`

The proposed wording of [#242](#) takes a stab at it but gets it wrong.

Proposed Resolution

Add two new paragraphs to [intro.defs] that read:

-?-

constant subexpression

expression whose evaluation as subexpression of a conditional-expression CE (ISO/IEC 14882:2014 §5.16) would not prevent CE from being a core constant expression (ISO/IEC 14882:2014 §5.19)

-?-

expression-equivalent

relationship that exists between two expressions E1 and E2 such that

- E1 and E2 have the same effects,
- `noexcept(E1) == noexcept(E2)`, and
- E1 is a constant subexpression if and only if E2 is a constant subexpression

[Editor's note: the definition of "constant subexpression" is taken from the latest C++17 draft.]

Change [iterator.utils.iter_move]/1 as follows:

```
1 The name iter_move denotes a customization point
object (3.3.2.3). The effect of the expression
+ object (3.3.2.3). The expression
  ranges::iter_move(E) for some expression E is
equivalent to the following:
+ expression-equivalent to the following:
(1.1) iter_move(E), if that expression is well formed
+(1.1) static_cast<decltype(iter_move(E))>(iter_move(E)),
+ if that expression is well-formed
  when evaluated in a context that does not include
  ranges::iter_move but does include the lookup set
  produced by argument-dependent lookup (ISO/IEC
  14882:2014 §3.4.2).
(1.2) Otherwise, if the expression *E is well formed
(1.2.1) If *E is an lvalue, std::move(*E).
(1.2.2) Otherwise, *E.
+(1.2) Otherwise, if the expression *E is well-formed:
+(1.2.1) if *E is an lvalue, std::move(*E);
+(1.2.2) otherwise, static_cast<decltype(*E)>(*E).
(1.3) Otherwise, ranges::iter_move(E) is ill-formed.
```

Change [iterator.utils.iter_swap]/1 as follows:

```
1 The name iter_swap denotes a customization point
object (3.3.2.3). The effect of the expression
+ object (3.3.2.3). The expression
  ranges::iter_swap(E1, E2) for some expressions E1
  and E2 is equivalent to the following:
+ and E2 is expression-equivalent to the following:
(1.1) (void)iter_swap(E1, E2), if that expression is well-
  formed when evaluated in a context that does not
  include ranges::iter_swap but does include the
  lookup set produced by argument-dependent lookup
  (ISO/IEC 14882:2014 §3.4.2) and the following
```

declaration:

```
void iter_swap(auto, auto) = delete;
```

(1.2) - Otherwise, if the types of E1 and E2 both satisfy Readable, and if the reference type of E1 is swappable with (4.2.11) the reference type of E2, then `ranges::swap(*E1, *E2)`.

(1.3) - Otherwise, if the types T1 and T2 of E1 and E2 satisfy `IndirectlyMovableStorable<T1, T2>()` && `IndirectlyMovableStorable<T2, T1>()`,

~~exchanges the values denoted by E1 and E2.~~

+ `(void)(*E1 = iter_exchange_move(E2, E1))`, except
+ that E1 is evaluated only once.

+2 `iter_exchange_move` is an exposition-only function specified as:

+ `template <class X, class Y>`
+ `constexpr value_type_t<remove_reference_t<X>> iter_exchange_move(X&& x, Y&& y)`
+ `noexcept(see below);`

+(2.1) Effects: Equivalent to:

+ `value_type_t<remove_reference_t<X>> old_value(iter_move(x));`
+ `*x = iter_move(y);`
+ `return old_value;`

+(2.2) The expression in the `noexcept` is equivalent to:

+ `NE(remove_reference_t<X>, remove_reference_t<Y>) &&`
+ `NE(remove_reference_t<Y>, remove_reference_t<X>)`

+ Where `NE(T1, T2)` is the expression:

+ `is_nothrow_constructible<value_type_t<T1>, rvalue_reference_t<T1>>::value &&`
+ `is_nothrow_assignable<value_type_t<T1>&, rvalue_reference_t<T1>>::value &&`
+ `is_nothrow_assignable<reference_t<T1>, rvalue_reference_t<T2>>::value &&`
+ `is_nothrow_assignable<reference_t<T1>, value_type_t<T2>>::value > &&`
+ `is_nothrow_move_constructible<value_type_t<T1>>::value &&`
+ `noexcept(ranges::iter_move(declval<T1>()))`

289: [iterator, count) ranges

- [alg.copy] Does `copy_n` need a “Requires:” element stating that `[result, result + n)` is a valid range?
- same concern for `fill_n` and `generate_n`.

We have “The result of the application of functions in the library to invalid ranges is undefined.” in [iterator.requirements.general]/10 where “range” is defined: “A range is an iterator and a sentinel that designate the beginning and end of the computation.” Instead of adding a requirement to every `_n` algorithm, we should expand the blanket wording to cover `[iterator, count)` ranges as well.

Proposed Resolution

Change [iterator.requirements.general] as follows:

7 Most of the library’s algorithmic templates that operate on data structures have interfaces that use ranges. A range is an iterator and a sentinel that designate the beginning and end of the computation
+ , or an iterator and a count that designate the beginning and the number of elements to which
+ the computation is to be applied.

8 An iterator and a sentinel denoting a range are comparable.
~~A sentinel denotes an element when it compares equal to an iterator i, and i points to that element.~~

The types of a sentinel and an iterator that denote a range must satisfy Sentinel (9.3.9). A range $[i, s)$ is empty if $i == s$; otherwise, $[i, s)$ refers to the elements in the data structure starting with the element pointed to by i and up to but not including the element pointed to by the first iterator j such that $j == s$.

- 9 A sentinel s is called reachable from an iterator i if and only if there is a finite sequence of applications of the expression $++i$ that makes $i == s$. If s is reachable from i ,
~~they denote a range.~~
+ $[i, s)$ denotes a range.
- + ? A counted range $[i, n)$ is empty if $n == 0$; otherwise, $[i, n)$ refers to the n elements in the data structure starting with the element pointed to by i and up to but not including the element pointed to by the result of incrementing i n times.
- 10 A range $[i, s)$ is valid if and only if s is reachable from i .
+ A counted range $[i, n)$ is valid if and only if $n == 0$; or n is positive, i is dereferenceable, and $[++i, --n)$ is valid.
The result of the application of functions in the library to invalid ranges is unspecified.

Change `[alg.fill]/p1` as follows:

1. Effects: `fill` assigns value through all the iterators in the range $[first, last)$.
~~`fill_n` assigns value through all the iterators in the range $[first, first + n)$ if n is~~
+ `fill_n` assigns value through all the iterators in the counted range $[first, n)$ if n is positive, otherwise it does nothing.

Change `[alg.generate]/p1-2` as follows:

- ~~1. Effects: Assigns the value of `invoke(gen)` through successive iterators in the range $[first, last)$, where `last` is `first + max(n, 0)` for `generate_n`.~~
+1. Effects: The `generate` algorithms invoke the function object `gen` and assign the return value of `gen` through all the iterators in the range $[first, last)$. The `generate_n` algorithm invokes the function object `gen` and assigns the return value of `gen` through all the iterators in the counted range $[first, n)$ if n is positive, otherwise it does nothing.
- ~~2. Returns: `last`~~
+2. Returns: `last`, where `last` is `first + max(n, 0)` for `generate_n`
- ~~3. Complexity: Exactly `last - first` evaluations of `invoke(gen)` and assignments.~~
+3. Complexity: Exactly `last - first` or n evaluations of `invoke(gen)` and assignments, respectively.

298: `common_iterator::operator->` with xvalue operator*

The specification states:

4 *Effects*: Given an object i of type I

(4.1) — if I is a pointer type or if the expression $i.operator->()$ is well-formed, this function returns `iter`.

(4.2) — Otherwise, if the expression $*iter$ is a glvalue, this function is equivalent to `return addressof(*iter);`

(4.3) — Otherwise, ...

`std::addressof` requires lvalue arguments, so the effects of 4.2 are ill-formed when $*iter$ is an xvalue.

Proposed Resolution:

Change [common.iter.op.star]/4.2 as follows:

```
4 Effects: Given an object i of type I
(4.1) – if I is a pointer type or if the expression i.operator->() is well-formed, this function
      returns iter.
(4.2) – Otherwise, if the expression *iter is a glvalue, this function is equivalent to return
      addressof(*iter);
+(4.2) – Otherwise, if the expression *iter is a glvalue, this function is equivalent to:
+   auto&& tmp = *iter;
+   return addressof(tmp);
(4.3) – Otherwise, [...]
```

300: Is it intended that an aggregate with a deleted or nonexistent default constructor satisfy DefaultConstructible?

For example, given

```
struct A{
    A(const A&) = default;
};
```

The following expressions are all valid:

```
A{};
new A{};
new A[5]{};
```

which would seem to mean that A meets DefaultConstructible as defined in [concepts.lib.object.defaultconstructible], unless I missed something.

If this is intended, then the note in [concepts.lib.object.defaultconstructible] is incorrect.

Proposed Resolution

Adopt [P0547R2: “Ranges TS: Assorted Object Concept Fixes”](#).

301: Is it intended that Constructible<int&, long&>() is true?

__BindableReference is defined with a function-style cast expression, which is equivalent to a C-style cast in the single-argument case, which means it can degenerate to a reinterpret_cast or an access-bypassing static_cast.

Proposed Resolution

Adopt [P0547R2: “Ranges TS: Assorted Object Concept Fixes”](#).

310: Movable<int&&>() is true and it should probably be false

Also see [#301](#)

Proposed Resolution

Adopt [P0547R2: “Ranges TS: Assorted Object Concept Fixes”](#).

313: MoveConstructible<T>() != std::is_move_constructible<T>()

...when T is const some_move_only_type due to the use of remove_cv_t in the definition of MoveConstructible. This is sensible if MoveConstructible means “Can I construct a T object from an rvalue expression with T’s value type?” but becomes very surprising if we align the other `structibles with the meaning of the standard type traits.

CopyConstructible needs to change similarly for consistency.

Proposed Resolution

Adopt [P0547R2: “Ranges TS: Assorted Object Concept Fixes”](#).

314: ConvertibleTo<T&&, U> should say something about the final state of the source object

Assignable and MoveConstructible explicitly say that the source object may end up in a moved-from state as a result of the operation. I think ConvertibleTo should also.

Proposed resolution

Fixed by the proposed resolution of [#167](#).

322: ranges::exchange should be constexpr and conditionally noexcept

Proposed Resolution

Adopt [P0579R1: “constexpr for <experimental/ranges/iterator>”](#).

338: common_reference doesn’t work with some proxy references

Consider:

```
namespace ranges = std::experimental::ranges;

struct MyIntRef {
    MyIntRef(int &);
};

using T = ranges::common_reference_t<int &, MyIntRef>; // doesn't work
using U = ranges::common_type_t<int &, MyIntRef>; // also doesn't work
```

I haven’t decided yet if this is a bug or not. It’s a bug. At least, it’s a bug for common_reference, not for common_type.

[Note: As of C++14 (IIRC), std::common_type<int &, MyIntRef>::type is MyIntRef because it doesn’t decay types before the ternary conditional test, so this is a regression wrt C++14. However, with C++17, std::common_type first decays types before the ternary conditional test, so we won’t be regressing behavior relative to C++17 if we don’t accommodate this case.—end note]

Discussion

Stepping through the specification of common_reference with the types int& and MyIntRef:

(3.3.1) — If `COMMON_REF(T1, T2)` is well-formed and denotes a reference type then the member typedef type denotes that type

This bullet is meant to handle reference types for which the built-in ternary operator yields another reference type (after some argument munging to avoid some weirdness with the ternary operator that causes premature type decay). The intention is to prevent the next bullet from firing, thereby preventing users from overriding the language rules regarding reference binding via the `basic_common_reference` customization point.

This rule doesn't fire for `int&` and `MyIntRef` because the logical common reference in this case is `MyIntRef`, which is not a reference type.

(Besides, this bullet should only be considered when `T1` and `T2` are reference types. We can tighten up the spec by requiring that condition. The issue under discussion doesn't require that change, but we propose it anyway.)

(3.3.2) — Otherwise, if `basic_common_reference<UNCVREF(T1), UNCVREF(T2), XREF(T1), XREF(T2)>::type` is well-formed, then the member typedef type denotes that type.

This bullet only fires when the user has specialized `basic_common_reference` on their argument types. Not relevant here.

(3.3.3) — Otherwise, if `common_type_t<T1, T2>` is well-formed, then the member typedef type denotes that type.

By dispatching to `common_type` we pick up any user specializations of that type trait. Trouble is, it decays types before looking for a common type. By decaying `int&` to `int`, it now becomes impossible to find the common type.

(3.3.4) — Otherwise, there shall be no member type.

Therefore, there is no common reference.

The fix is to, before testing for `common_type`, first put the two types in a ternary conditional *without* decaying and see if that results in a well-formed type. Only if it doesn't do we fall back to `common_type`.

The proposed resolution is expressed in terms of the `COND_RES(X, Y)` pseudo-macro, which was defined as `decltype(declval<bool>() ? declval<X>() : declval<Y>())`. This macro is buggy because `COND_RES(int, int)` comes out as `int&&` instead of `int`. That's because the type of `declval<int>()` is `int&&`. To make it work, `COND_RES(X, Y)` needs to be redefined as the somewhat cryptic `decltype(declval<bool>() ? declval<X(&())>() : declval<Y(&())>())`. This formulation perfectly preserves the value category of the arguments.

Proposed Resolution

Change `[meta.trans.other]/p1` as follows:

1.
Let `CREF(A)` be `add_lvalue_reference_t<const remove_reference_t<A>>`.
Let `UNCVREF(A)` be `remove_cv_t<remove_reference_t<A>>`. Let `XREF(A)` denote a unary template `T` such that `T<UNCVREF(A)>` denotes the same type as `A`. Let `COPYCV(FROM, TO)` be an alias for type `TO` with the addition of `FROM`'s top-level cv-qualifiers. [Example: `COPYCV(const int, volatile short)` is an alias for `const volatile short`. —end example] Let `RREF_RES(Z)` be `remove_reference_t<Z>&&` if `Z` is a reference type or `Z` otherwise. Let `COND_RES(X, Y)` be
~~`decltype(declval<bool>() ? declval<X>() : declval<Y>())`.~~
`+decltype(declval<bool>() ? declval<X(&())>() : declval<Y(&())>())`.
Given types `A` and `B`, let `X` be `remove_reference_t<A>`, let `Y` be `remove_reference_t<B>`, and let `COMMON_REF(A, B)` be:

Change the description of `common_reference` [meta.trans.other]/p3 as follows:

3
For the `common_reference` trait applied to a parameter pack `T` of types, the member type shall be either defined or not present as follows:
(3.1) – If `sizeof...(T)` is zero, there shall be no member type.
(3.2) – Otherwise, if `sizeof...(T)` is one, let `T1` denote the sole type in the pack `T`. The member typedef type shall denote the same type as `T1`.
(3.3) – Otherwise, if `sizeof...(T)` is two, let `T1` and `T2` denote the two types in the pack `T`. Then
~~(3.3.1) – If `COMMON_REF(T1, T2)` is well-formed and denotes a reference~~
+(3.3.1) – If `T1` and `T2` are reference types and `COMMON_REF(T1, T2)` is
+ **well-formed and denotes a reference**
type then the member typedef type denotes that type.
(3.3.2) – Otherwise, if `basic_common_reference<UNCVREF(T1), UNCVREF(T2), XREF(T1), XREF(T2)>::type` is well-formed, then the member typedef type denotes that type.
+(3.3.x) – Otherwise, if `COND_RES(T1, T2)` is well-formed, then the
+ **member typedef type denotes that type.**
(3.3.3) – Otherwise, if `common_type_t<T1, T2>` is well-formed, then the member typedef type denotes that type.
(3.3.4) – Otherwise, there shall be no member type.

339: After P0547R0, const-qualified iterator types are not Readable or Writable

Readable requires Movable. const-qualified types are not Movable after P0547R0. Ouch.

Proposed Resolution

Adopt [P0547R2: “Ranges TS: Assorted Object Concept Fixes”](#).

340: GB 1 (001): Consider all outstanding issues before the final TS is produced

General Comment

Consider all outstanding issues before the final TS is produced.

Proposed Resolution

All outstanding issues have been so considered, and all priority 1 and priority 2 issues resolved.

361: P0541 is missing semantics for `OutputIterator`’s writable post-increment result

[P0541](#) changes the definition of `OutputIterator` to:

```
template <class I, class T>
concept bool OutputIterator() {
    return Iterator<I>() && Writable<I, T>() &&
        requires(I i, T&& t) {
            *i++ = std::forward<T>(t); // not required to be equality preserving
        };
}
```

It notably fails to add any semantic constraints on the expression `*i++ = std::forward<T>(t)`. What does it do? Who knows?

Proposed Resolution

After [iterators.output]/p1 and before p2, add the following:

- x. Let E be an expression such that `decltype(E)` is T , and let i be a dereferenceable object of type I . Then `OutputIterator<I, T>()` is satisfied only if `*i++ = E`; has effects equivalent to:

```
*i = E;  
++i;
```

366: `common_iterator::operator->` is underconstrained

As specified in #318. The “proxy case” constructs an object of type `value_type_t<I>` from `reference_t<I>` but does not require `Constructible<value_type_t<I>, reference_t<I>>()`.

Proposed Resolution

Change the class synopsis of `common_iterator` ([common.iterator]) as follows (includes part of the resolution of #368):

```
see below operator*();  
see below operator*() const requires dereferenceable<I const>;  
see below operator->() const requires Readable<I const>();  
+see below operator->() const requires see below;
```

To [common.iter.op.ref], add a p3 that reads:

1. The expression in the requires clause is equivalent to:

```
Readable<I const>() &&  
(requires (const I& i) { i.operator->(); } ||  
is_reference<reference_t<I>>::value ||  
Constructible<value_type_t<I>, reference_t<I>>())
```

367: `advance`, `distance`, `next`, and `prev` should be customization point objects

Imagine passing a new-style `RandomAccessIterator` I for which `reference_t<I>` is not a reference type to (unqualified) `advance`. Further imagine that I has namespace `std` as an associated namespace (by wrapping a `std` iterator, for instance). Then `std::advance` might get called, which will treat the iterator as an `InputIterator` (due to category demotion), doing N increments instead of just one `+=`.

Making `ranges::advance` a CPO could help in some scenarios. Ditto for `distance`, `next`, and `prev`.

Proposed Resolution

[Editor’s note: this wording assumes that the resolution of #256 has been applied. It depicts distinct overloads for `next(i)` and `next(i, n)` instead of the semantically equivalent `next(i, n = 1)` to allow implementations that

strengthen noexcept to conformingly reflect noexcept(++i) for the next(i) case. Otherwise, the “new” wording is as close as possible to the “old” wording to minimize review costs: it simply turns function template overload sets into member function template overload sets.]

Change the synopsis of <experimental/ranges/iterator> in [iterator.synopsis] as follows:

```
// 9.6.4, iterator operations:
template <Iterator I>
void advance(I& i, difference_type_t<I> n);
template <Iterator I, Sentinel<I> S>
void advance(I& i, S bound);
template <Iterator I, Sentinel<I> S>
difference_type_t<I> advance(I& i, difference_type_t<I> n, S bound);
template <Iterator I, Sentinel<I> S>
difference_type_t<I> distance(I first, S last);
template <Iterator I>
I next(I x, difference_type_t<I> n = 1);
template <Iterator I, Sentinel<I> S>
I next(I x, S bound);
template <Iterator I, Sentinel<I> S>
I next(I x, difference_type_t<I> n, S bound);
template <BidirectionalIterator I>
I prev(I x, difference_type_t<I> n = 1);
template <BidirectionalIterator I>
I prev(I x, difference_type_t<I> n, I bound);
+ namespace {
+   constexpr unspecified advance = unspecified;
+   constexpr unspecified distance = unspecified;
+   constexpr unspecified next = unspecified;
+   constexpr unspecified prev = unspecified;
+ }

[...]
```

```
template <Iterator I>
counted_iterator<I> make_counted_iterator(I i, difference_type_t<I> n);

template <Iterator I>
void advance(counted_iterator<I>& i, difference_type_t<I> n);

// 9.7.8, unreachable sentinels:

[...]
```

```
// 9.11, range primitives:
namespace {
    constexpr unspecified size = unspecified ;
    constexpr unspecified empty = unspecified ;
    constexpr unspecified data = unspecified ;
    constexpr unspecified cdata = unspecified ;
}
template <Range R>
difference_type_t<iterator_t<R>> distance(R&& r);
template <SizedRange R>
difference_type_t<iterator_t<R>> distance(R&& r);
```

In [iterator.operations], change paragraph 1 as follows:

1 Since only types that satisfy RandomAccessIterator provide the + operator, and types that satisfy SizedSentinel provide the - operator, the library provides four

~~function templates~~
+ customization point objects
advance, distance, next, and prev. These

~~function templates~~
+ customization point objects

use + and - for random access iterators and ranges that satisfy SizedSentinel

, respectively

(and are, therefore, constant time for them); for output, input, forward and bidirectional iterators they use ++ to provide linear time implementations.

and replace the remainder of [iterator.operations] entirely with:

2 The name `advance` denotes a customization point object (`\ref{customization.point.object}`). It has the following function call operators:

```
template <Iterator I>
constexpr void operator()(I& i, difference_type_t<I> n) const;
```

3 *Requires*: `n` shall be negative only for bidirectional iterators.

4 *Effects*: For random access iterators, equivalent to `i += n`. Otherwise, increments (or decrements for negative `n`) iterator `i` by `n`.

```
template <Iterator I, Sentinel<I> S>
constexpr void operator()(I& i, S bound) const;
```

5 *Requires*: If `Assignable<I&, S>()` is not satisfied, `[i, bound)` shall denote a range.

6 *Effects*:

(6.1) — If `Assignable<I&, S>()` is satisfied, equivalent to `i = std::move(bound)`.

(6.2) — Otherwise, if `SizedSentinel<S, I>()` is satisfied, equivalent to `advance(i, bound - i)`.

(6.3) — Otherwise, increments `i` until `i == bound`.

```
template <Iterator I, Sentinel<I> S>
constexpr difference_type_t<I> operator()(I& i, difference_type_t<I> n, S bound) const;
```

7 *Requires*: If `n > 0`, `[i, bound)` shall denote a range. If `n == 0`, `[i, bound)` or `[bound, i)` shall denote a range. If `n < 0`, `[bound, i)` shall denote a range and `(BidirectionalIterator<I>() && Same<I, S>())` shall be satisfied.

8 *Effects*:

(8.1) — If `SizedSentinel<S, I>()` is satisfied:

(8.1.1) — If `|n| >= |bound - i|`, equivalent to `advance(i, bound)`.

(8.1.2) — Otherwise, equivalent to `advance(i, n)`.

(8.2) — Otherwise, increments (or decrements for negative `n`) iterator `i` either `n` times or until `i == bound`, whichever comes first.

9 *Returns*: `n - M`, where `M` is the distance from the starting position of `i` to the ending position.

10 The name `distance` denotes a customization point object. It has the following function call operators:

```
template <Iterator I, Sentinel<I> S>
constexpr difference_type_t<I> operator()(I first, S last) const;
```

9 *Requires*: `[first, last)` shall denote a range, or `(Same<S, I>() && SizedSentinel<S, I>())` shall be satisfied and `[last, first)` shall denote a range.

10 *Effects*: If `SizedSentinel<S, I>()` is satisfied, returns `(last - first)`; otherwise, returns the number of increments needed to get from `first` to `last`.

```
template <Range R>
constexpr difference_type_t<iterator_t<R>> operator()(R&& r) const;
```

11 *Effects*: Equivalent to: `return distance(ranges::begin(r), ranges::end(r));`
`(ref{iterator.range})`

[*Editor's note*: Include the following paragraph only if the PR of [#211](#) has been applied.]

12 *Remarks*: Instantiations of this function template may be ill-formed if the declarations in `<experimental/range/range>` are not in scope at the point of instantiation (`\cxxref{temp.point}`).

```
template <SizedRange R>
constexpr difference_type_t<iterator_t<R>> operator()(R&& r) const;
```

13 *Effects*: Equivalent to: `return ranges::size(r);` (`\ref{range.primitives.size}`)

[*Editor's note*: Include the following paragraph only if the PR of [#211](#) has been applied.]

14 *Remarks*: Instantiations of this function template may be ill-formed if the declarations in `<experimental/range/range>` are not in scope at the point of instantiation (`\cxxref{temp.point}`).

15 The name `next` denotes a customization point object. It has the following function call operators:

```
template <Iterator I>
constexpr I operator()(I x) const;
```

16 *Effects*: Equivalent to: `++x; return x;`

```
template <Iterator I>
constexpr I operator()(I x, difference_type_t<I> n) const;
```

17 *Effects*: Equivalent to: `advance(x, n); return x;`

```
template <Iterator I, Sentinel<I> S>
constexpr I operator()(I x, S bound) const;
```

18 *Effects*: Equivalent to: `advance(x, bound); return x;`

```
template <Iterator I, Sentinel<I> S>
constexpr I operator()(I x, difference_type_t<I> n, S bound) const;
```

19 *Effects*: Equivalent to: `advance(x, n, bound); return x;`

20 The name `prev` denotes a customization point object. It has the following function call operators:

```
template <BidirectionalIterator I>
constexpr I operator()(I x) const;
```

21 *Effects*: Equivalent to: `--x; return x;`

```
template <BidirectionalIterator I>
constexpr I operator()(I x, difference_type_t<I> n) const;
```

22 *Effects*: Equivalent to: `advance(x, -n); return x;`

```
template <BidirectionalIterator I, Sentinel<I> S>
constexpr I operator()(I x, difference_type_t<I> n, S bound) const;
```

23 *Effects*: Equivalent to: `advance(x, -n, bound); return x;`

Also strike the declaration of the `advance` overload from the `counted_iterator` synopsis in `[counted.iterator]`, and strike its specification: paragraphs 9 and 10 in `[counted.iter.nonmember]`.

Strike paragraphs 1 & 2 from `[range.primitives]`:

```
template <Range R>
difference_type_t<iterator_t<R>> distance(R&& r);

1 Effects: Equivalent to: return ranges::distance(ranges::begin(r), ranges::end(r));

template <SizedRange R>
difference_type_t<iterator_t<R>> distance(R&& r);

2 Effects: Equivalent to: return ranges::size(r);
```

368: `common_iterator`'s and `counted_iterator`'s `const operator*` need to be constrained

Some iterators (e.g., `insert_iterator`) don't have a `const`-qualified `operator*`. When wrapping such an iterator, `counted_iterator` and `common_iterator` also should not have such an overload of `operator*`.

Proposed Resolution

1. Change the `common_iterator` class synopsis `[common.iterator]` as follows:

```
see below operator*();
see below operator*() const;
see below operator->() const requires Readable<I>();
+see below operator*() const requires dereferenceable<I> const>;
+see below operator->() const requires Readable<I> const>();
```

```
common_iterator& operator++();
common_iterator operator++(int);
```

1. Change `common_iterator::operator*` `[common.iter.op.star]` as follows:

```
decltype(auto) operator*();
decltype(auto) operator*() const;
```

```
+decltype(auto) operator*() const requires dereferenceable<I const>;
1 Requires: !is_sentinel
2 Effects: Equivalent to: return *iter;
```

1. Change `common_iterator::operator->` [common.iter.op.ref] as follows:

```
see below operator->() const requires Readable<I>();
+see below operator->() const requires Readable<I const>();
1 Requires: !is_sentinel
2 Effects: Given an object i of type I
...
```

1. Change the `counted_iterator` class synopsis [counted.iterator] as follows:

```
see below operator*();
see below operator*() const;
+see below operator*() const requires dereferenceable<I const>;

counted_iterator& operator++();
counted_iterator operator++(int);
```

1. Change `counted_iterator::operator*` [counted.iter.op.star] as follows:

```
decltype(auto) operator*();
decltype(auto) operator*() const;
+decltype(auto) operator*() const requires dereferenceable<I const>;
1 Effects: Equivalent to: return *current;
```

379: Switch to variable concepts

I’m getting tired of typing and looking at `()` everywhere. I suggest switching whole hog to variable templates. I threatened to do that in the committee and nobody objected.

Bike Shedding!

The change would involve renaming concept “overloads” like the cross-type `EqualityComparable`. Following the existing practice of `is_swappable`/`is_swappable_with`, the sensible choice is `EqualityComparableWith`. Another satisfactory choice would be `EqualityComparableTo`.

I suggest the suffix `With` for cross-type concepts that are inherently symmetric in nature. The suffix `To` or `Of` could be used for asymmetric cross-type concepts like `ConvertibleTo`.

Possibly Related

Along that vein, should we consider renaming `Sentinel` to `SentinelOf`? This reads nicer:

```
template <InputIterator I, SentinelOf<I> S>
void algorithm( I, S );
```

Proposed Resolution

Adopt [P0651R1: “Ranges TS: Switch the Ranges TS to Use Variable Concepts”](#).

381: Readable types with prvalue reference types erroneously model Writable

Consider:

```
struct MakeString
{
    using value_type = std::string;
    std::string operator*() const
    {
        return std::string();
    }
};

static_assert(!Writable<MakeString, std::string>()); // FAILS
```

This is a huge usability problem, since it permits users to, e.g., pass a range to sort that is not, in fact, sortable. The Ranges TS inherited this problem from the Palo Alto Report ([N3351](#)), which also has this bug. Fixing this will be **tricky** easy :-) without messing up proxy iterators.

EDIT: See [ericniebler/range-v3#573](#).

Resolution Discussion:

One fix would be to require that `*o` return a true reference, but that breaks when `*o` returns a proxy reference. The trick is in distinguishing between a prvalue that is a proxy from a prvalue that is just a value. The trick lies in recognizing that a proxy always represents a (logical, if not physical) indirection. As such, adding a `const` to the proxy should not effect the mutability of the thing being proxied. Further, if `decltype(*o)` is a true reference, then adding `const` to it has no effect, which also does not effect the mutability. So the fix is to add `const` to `decltype(*o)`, `const_cast *o` to that, and *then* test for writability.

Proposed Resolution

Adopt [P0547R2: “Ranges TS: Assorted Object Concept Fixes”](#).

382: Don’t try to forbid overloaded & in Destructible

Per LWG Kona consensus.

Proposed Resolution

(Relative to P0547R0) Change `[concepts.lib.object.destructible]` as follows:

```
template <class T>
concept bool Destructible() {
+ return is_nothrow_destructible<T>::value; // see below
- return is_nothrow_destructible<T>::value && // see below
    requires(T& t, const remove_reference_t<T>& ct) {
    { &t } -> Same<remove_reference_t<T>*>&& // not required to be equality preserving
    { &ct } -> Same<const remove_reference_t<T>*>&& // not required to be equality preserving
    };
}
```

Strike `[concepts.lib.object.destructible]/p2` (“The expression requirement `&ct ...`”) and `[concepts.lib.object.destructible]/p3` (“n a (possibly const) lvalue `t` of type...”).

386: P0541: basic exception guarantee in counted_iterator's postincrement

Comment from LWG Kona review:

basic guarantee in counted_iterator's postincrement and postdecrement. (If that results in inconsistently ordered operations, STL wants a note to explain why.)

These operators are specified to update the count member and then directly return the result of postincrementing/postdecrementing the iterator member. My concern was that the count and iterator members could become desynchronized if the iterator operation throws. We've avoided the issue in the past by updating the count after updating the iterator.

The real issue here is that we would like for adaptors to preserve whatever exception guarantee the adapted type provides. This is trivially the case for reverse_iterator, move_iterator, and I think common_iterator, but counted_iterator needs to try a little harder or explicitly warn users that counted_iterator's postincrement operation for single-pass iterators doesn't maintain the guarantee of the adapted iterator.

Proposed Resolution

Wording relative to D0541R1. Change the specification of operator++(int):

```
decltype(auto) operator++(int);
```

4 Requires: cnt > 0.

5 Effects: Equivalent to:

```
--cnt  
return current++;  
+try { return current++; }  
+catch(...) { ++cnt; throw; }
```

387: writable should work with rvalues

During LWG Kona review of D0547R1, writable is defined therein as:

```
template <class Out, class T>  
concept bool Writable() {  
    return requires(Out& o, T&& t) {  
        *o = std::forward<T>(t);  
    };  
}
```

The concept only requires writability for lvalues, but it seems reasonable to require that writability is oblivious to value category.

Proposed Resolution

Adopt [P0547R2: "Ranges TS: Assorted Object Concept Fixes"](#).

396: SizedRange should not require size() to be callable on a const qualified object

See discussion of [ericniebler/range-v3#385](#) and [discussion on Slack](#).

Motivating example: range-v3 drop_while. begin must determine and cache the initial iterator value to satisfy the amortized O(1) complexity requirement, and so needs to mutate non-observable state inside the view object. If the adapted view has iterators that satisfy SizedSentinel, drop_while could implement size by calling begin and end and returning their difference. Transitively, doing so would require size to be a non-const member as well.

Proposed Resolution

```
template <class T>
concept bool SizedRange() {
    return Range<T>() &&
        !disable_sized_range<remove_cv_t<remove_reference_t<T>>> &&
requires(const remove_reference_t<T>& t) {
+ requires(T& t) {
    { ranges::size(t) } -> ConvertibleTo<difference_type_t<iterator_t<T>>>;
};
}
```

2 Given an lvalue t of type remove_reference_t<T>, SizedRange<T>() is satisfied if and only if:

~~(2.1) size(t) returns the number of elements in t.~~
+ (2.1) ranges::size(t) is O(1), does not modify t, and is equal to ranges::distance(t).

398: Assignable semantic constraints contradict each other for self-assign

As of P0547, we have this for the post-conditions on Assignable's t = std::forward<U>(u) expression:

(1.2.1) – t is equal to u2.

(1.2.2) – If u is a non-const xvalue, the resulting state of the object to which it refers is unspecified.

If &t == &u and u is a non-const xvalue, is the resulting state u2 or unspecified?

Proposed Resolution

This wording is relative to [D0547R1](#).

Change paragraph 1 of [concepts.lib.corelang.assignable] (7.3.10) as follows:

```
1. Let t be an lvalue
which
+ that
    refers to an object o such that decltype((t)) is T, and u an expression
    such that decltype((u)) is U. Let u2 be a distinct object that is equal to
    u. Then Assignable<T, U>() is satisfied if and only if
    (1.1) – std::addressof(t = u) == std::addressof(o).
    (1.2) – After evaluating t = u:
(1.2.1) – t is equal to u2.
+ (1.2.1) – t is equal to u2, unless u is a non-const xvalue that refers to o.
    (1.2.2) – If u is a non-const xvalue, the resulting state of the object to which it
refers is unspecified. [ Note: the object must still meet the requirements of
the library component that is using it. The operations listed in those
requirements must work as specified. end note ]
+ refers is valid but unspecified ([lib.types.movedfrom]).
    (1.2.3) – Otherwise, if u is a glvalue, the object to which it refers is not modified.

2. There is no
+ 2. There need not be a
    subsumption relationship between Assignable<T, U>() and is_lvalue_reference<T>::value.

+ 3. [Note: Assignment need not be a total function ([structure.requirements]); in particular,
```

```
+ if assignment to an object x can result in a modification of some other object y, then
+ x = y is likely not in the domain of =. -- end note]
```

399: iterators that return move-only types by value do not satisfy Readable

The problem is with `CommonReference<reference_t<I>, value_type_t<I>&>()`. Imagine an iterator `I` such that `reference_t<I>` is `std::unique_ptr<int>`. The `CommonReference` first computes the common reference type to be `std::unique_ptr<int>`, then it tests that `std::unique_ptr<int>&` is `ConvertibleTo std::unique_ptr<int>`, which is of course false.

The fix is to instead be testing `CommonReference<reference_t<I>&&, value_type_t<I>&>()`. That causes the common reference to be computed as `const std::unique_ptr<int>&`, and `std::unique_ptr<int>&` is indeed convertible to that.

Proposed Resolution

(Includes the resolution for <https://github.com/ericniebler/stl2/issues/339>, accepted by LWG in Kona but not yet moved in full committee).

Change the `Readable` concept [iterators.readable] as follows:

```
template <class I>
concept bool Readable() {
    return requires {
        typename value_type_t<I>;
        typename reference_t<I>;
        typename rvalue_reference_t<I>;
    } &&
    CommonReference<reference_t<I>, value_type_t<I>&>() &&
    CommonReference<reference_t<I>, rvalue_reference_t<I>>>() &&
    CommonReference<rvalue_reference_t<I>, const value_type_t<I>&>();
    + CommonReference<reference_t<I>&&, value_type_t<I>&>() &&
    + CommonReference<reference_t<I>&&, rvalue_reference_t<I>&&>() &&
    + CommonReference<rvalue_reference_t<I>&&, const value_type_t<I>&>();
}
```

404: Normative content in informative subclause

The entire subclause 6.2 [description] is informative, but it contains things like [customization.point.object] and the last few paragraphs of [structure.requirements] that are clearly intended to be normative rather than informative.

Proposed Resolution

Strike [structure.requirements]/p8, which states “If the semantic requirements of a declaration are not satisfied at the point of use, the program is ill-formed, no diagnostic required.”

Add a new subsection after “Requires paragraph” ([res.on.required]) called “Semantic requirements” (stable name [res.on.requirements]) with the following:

6.3.4.7 Semantic requirements [res.on.requirements]

1 If the semantic requirements of a declaration’s constraints~(\ref{structure.requirements}) are not satisfied at the point of use, the program is ill-formed, no diagnostic required.

Move subsection “Customization Point Objects” ([customization.point.object]) to a new subsection under “Conforming implementations” ([conforming]).

407: istreambuf_iterator::operator->

[LWG 2790](#) recently removed `istreambuf_iterator::operator->` from C++17. Should it be removed here as well?

Proposed Resolution

1. Remove the note in paragraph 1 of 9.8.3 [istreambuf.iterator]:

The class template `istreambuf_iterator` defines an input iterator (9.3.11) that reads successive *characters* from the streambuf for which it was constructed. `operator*` provides access to the current input character, if any. ~~[Note: `operator->` may return a proxy. — end note]~~ Each time `operator++` is evaluated, the iterator advances to the next input character. [...]

1. Remove the member from the class synopsis in 9.8.3 [istreambuf.iterator]:

```
charT operator*() const;
pointer operator->() const;
istreambuf_iterator& operator++();
proxy operator++(int);
```

411: find_first_of and mismatch should use IndirectRelation instead of Indirect(Binary)Predicate

Why do `find_first_of` and `mismatch` use `Indirect(Binary)Predicate` instead of `IndirectRelation`?

According to [cppreference.com](#), there are many algorithms that merely expect a `BinaryPredicate` and not a so-called `Compare ...` which is the current standard's name for `Relation`. In the Ranges TS, only `find_first_of` and `mismatch` expect `Predicates`; all the others expect `Relations`. Why?

I believe we picked this up from the Palo Alto report. Was the change intentional? Are the changed algorithms now over-constrained by requiring `Relations` whereas they could make do with `Predicates`?

Perhaps the authors of N3351 can give us the backstory on why that paper strengthened the requirements of some algorithms, but mysteriously not `find_first_of` and `mismatch`.

Proposed Resolution

Change `<experimental/ranges/algorithm>` synopsis ([algorithms.general]) as follows:

```
template <InputIterator I1, Sentinel<I1> S1, ForwardIterator I2, Sentinel<I2> S2,
class Proj1 = identity, class Proj2 = identity,
IndirectPredicate<projected<I1, Proj1>, projected<I2, Proj2>> Pred = equal_to<>>
+ IndirectRelation<projected<I1, Proj1>, projected<I2, Proj2>> Pred = equal_to<>>
I1
find_first_of(I1 first1, S1 last1, I2 first2, S2 last2,
Pred pred = Pred{},
Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{});

template <InputRange Rng1, ForwardRange Rng2, class Proj1 = identity,
class Proj2 = identity,
IndirectPredicate<projected<iterator_t<Rng1>, Proj1>,
projected<iterator_t<Rng2>, Proj2>> Pred = equal_to<>>
+ IndirectRelation<projected<iterator_t<Rng1>, Proj1>,
projected<iterator_t<Rng2>, Proj2>> Pred = equal_to<>>
safe_iterator_t<Rng1>
find_first_of(Rng1&& rng1, Rng2&& rng2,
Pred pred = Pred{},
Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{});
```

[...]

```
template <InputIterator I1, Sentinel<I1> S1, InputIterator I2, Sentinel<I2> S2,
        class Proj1 = identity, class Proj2 = identity,
        IndirectPredicate<projected<I1, Proj1>, projected<I2, Proj2>> Pred = equal_to<>>
+       IndirectRelation<projected<I1, Proj1>, projected<I2, Proj2>> Pred = equal_to<>>
        tagged_pair<tag::in1(I1), tag::in2(I2)>
        mismatch(I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = Pred{},
                Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{});

template <InputRange Rng1, InputRange Rng2,
        class Proj1 = identity, class Proj2 = identity,
        IndirectPredicate<projected<iterator_t<Rng1>, Proj1>,
        +       IndirectRelation<projected<iterator_t<Rng1>, Proj1>,
        projected<iterator_t<Rng2>, Proj2>> Pred = equal_to<>>
        tagged_pair<tag::in1(safe_iterator_t<Rng1>),
                tag::in2(safe_iterator_t<Rng2>)>
        mismatch(Rng1&& rng1, Rng2&& rng2, Pred pred = Pred{},
                Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{});
```

Make the same changes to the signatures of `find_first_of` and `mismatch` in sections “Find first of” ([`alg.find.first.of`]) and “Mismatch” ([`mismatch`]).

Make the following change to section “Range-and-a-half algorithms” ([`depr.algo.range-and-a-half`]) in Annex A:

```
template <InputIterator I1, Sentinel<I1> S1, InputIterator I2,
        class Proj1 = identity, class Proj2 = identity,
        IndirectPredicate<projected<I1, Proj1>, projected<I2, Proj2>> Pred = equal_to<>>
+       IndirectRelation<projected<I1, Proj1>, projected<I2, Proj2>> Pred = equal_to<>>
        tagged_pair<tag::in1(I1), tag::in2(I2)>
        mismatch(I1 first1, S1 last1, I2 first2, Pred pred = Pred{},
                Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{});

template <InputRange Rng1, InputIterator I2,
        class Proj1 = identity, class Proj2 = identity,
        IndirectPredicate<projected<iterator_t<Rng1>, Proj1>,
        +       IndirectRelation<projected<iterator_t<Rng1>, Proj1>,
        projected<I2, Proj2>> Pred = equal_to<>>
        tagged_pair<tag::in1(safe_iterator_t<Rng1>), tag::in2(I2)>
        mismatch(Rng1&& rng1, I2 first2, Pred pred = Pred{},
                Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{});
```

414: Remove `is_[nothrow]_indirectly_(movable|swappable)`

They are unused, after `iter_exchange` in [#284](#) is rewritten to be exposition-only.

Proposed Resolution

Presumes that the resolution of [#284](#) has been applied.

Modify the synopsis of `<experimental/ranges/iterator>` in [`iterator.synopsis`] as follows:

```
// 9.6, primitives:
// 9.6.1, traits:
template <class I1, class I2> struct is_indirectly_movable;
template <class I1, class I2 = I1> struct is_indirectly_swappable;
template <class I1, class I2> struct is_nothrow_indirectly_movable;
template <class I1, class I2 = I1> struct is_nothrow_indirectly_swappable;
template <class I1, class I2> constexpr bool is_indirectly_movable_v
```

```

= is_indirectly_movable<I1, I2>::value;
template <class I1, class I2> constexpr bool is_indirectly_swappable_v
= is_indirectly_swappable<I1, I2>::value;
template <class I1, class I2> constexpr bool is_nothrow_indirectly_movable_v
= is_nothrow_indirectly_movable<I1, I2>::value;
template <class I1, class I2> constexpr bool is_nothrow_indirectly_swappable_v
= is_nothrow_indirectly_swappable<I1, I2>::value;

template <class Iterator> using iterator_traits = see below;

template <Readable T> using iter_common_reference_t
    = common_reference_t<reference_t<T>, value_type_t<T>&>;

```

Strike paragraph [iterator.traits]/1 that begins: “The class templates `is_indirectly_movable`, `is_nothrow_indirectly_movable`, `is_indirectly_swappable`, and `is_nothrow_indirectly_swappable` shall be defined as follows” including the definitions of the named traits.

[420](#): Harmonize `common_type` with C++17’s [meta.trans.other]/p4

Looks like the language drifted since we lifted it for the PR of [#235](#). C++17’s [meta.trans.other]/p4 now reads:

Note B: Notwithstanding the provisions of (\cxxref{meta.type.synop}), and pursuant to \cxxref{namespace.std}, a program may specialize `common_type<T1, T2>` for types `T1` and `T2` such that `is_same_v<T1, decay_t<T1>>` and `is_same_v<T2, decay_t<T2>>` are each true. [*Note*: Such specializations are needed when only explicit conversions are desired between the template arguments. — *end note*] Such a specialization need not have a member named `type`, but if it does, that member shall be a *typedef-name* for an accessible and unambiguous cv-unqualified non-reference type `C` to which each of the types `T1` and `T2` is explicitly convertible. Moreover, `common_type_t<T1, T2>` shall denote the same type, if any, as does `common_type_t<T2, T1>`. No diagnostic is required for a violation of this Note’s rules.

This is much better, IMO.

Proposed Resolution

Replace our Note B (see [#235](#)) with the one in the current IS draft, quoted above.

In addition, make the following change to the (newly added) paragraph about `basic_common_reference` (see [#235](#)):

```

A program may specialize the basic_common_reference trait for
two cv unqualified non reference types if at least one of them depends on a
user defined type. Such a specialization need not have a member named
type.
+Notwithstanding the provisions of \cxxref{meta.type.synop}, and pursuant to
+\cxxref{namespace.std}, a program may specialize
+basic_common_reference<T, U, TQual, UQual> for types T and U such that
+is_same_v<T, decay_t<T>> and is_same_v<U, decay_t<U>> are each true.
+[ Note: Such specializations are needed when only explicit conversions are
+desired between the template arguments. —end note ] Such a specialization
+need not have a member named type, but if it does, that member shall be a
+typedef-name for an accessible and unambiguous type C to which each of the
+types TQual<T> and UQual<U> is convertible. Moreover,
+basic_common_reference<T, U, TQual, UQual>::type shall denote the same type,
+if any, as does basic_common_reference<U, T, UQual, TQual>::type. A program
+may not specialize basic_common_reference on the third or fourth parameters,
+TQual or UQual. No diagnostic is required for a violation of these rules.

```

[421](#): With `common_type`, the Ranges TS requires vendors to break conformance

The Ranges TS requires changes to `common_type` in namespace `std`. This is a bad idea. It requires vendors to break conformance in order to support the Ranges TS. Reformulate things so that we can move the `Common[Reference]` machinery out of namespace `std` and into namespace `std::experimental::ranges`.

This moves the last piece of the Ranges TS out of namespace `std`, making the TS a pure extension.

Proposed Resolution

[*Editorial*: Issue [#259](#) moves the `is_swappable` traits out of namespace `std` and into namespace `std::experimental::ranges`. This PR does the same for the `common_type` traits, leaving nothing left in `[meta.type.synop]`. This wording assumes the wording in (ready) issue [#259](#).]

Remove section “Header <type_traits> synopsis” (`[meta.type.synop]`).

Give section “Header <experimental/ranges/type_traits> synopsis (`[meta.type.synop.rng]`) the stable name `[meta.type.synop]` and make the following change to the synopsis:

```
namespace std { namespace experimental { namespace ranges { inline namespace v1 {
// (REF), type properties:
template <class T, class U> struct is_swappable_with;
template <class T> struct is_swappable;

template <class T, class U> struct is_nothrow_swappable_with;
template <class T> struct is_nothrow_swappable;

+// (REF), other transformations
+template <class... T> struct common_type;
+template <class T, class U, template <class> class TQual, template <class> class UQual>
+ struct basic_common_reference { };
+template <class... T> struct common_reference;

+template <class... T>
+ using common_type_t = typename common_type<T...>::type;
+template <class... T>
+ using common_reference_t = typename common_reference<T...>::type;

template <class T, class U> constexpr bool is_swappable_with_v
    = is_swappable_with<T, U>::value;
template <class T> constexpr bool is_swappable_v
    = is_swappable<T>::value;

template <class T, class U> constexpr bool is_nothrow_swappable_with_v
    = is_nothrow_swappable_with<T, U>::value;
template <class T> constexpr bool is_nothrow_swappable_v
    = is_nothrow_swappable<T>::value;
}}}}
```

Rename section “Additional type properties” (`[meta.unary.prop.rng]`, [#259](#)) to “Type properties” with the stable name `[meta.unary.prop]`.

Move section “Other transformations” (`[meta.trans.other]`) after “Type properties”.

Strike the editorial notes in `[meta.trans.other]` that read “Change Table 57 – “Other Transformations” in ISO/IEC 14882:2014 as follows”, and “Delete `[meta.trans.other]/p3` from ISO/IEC 14882:2014 and replace it with the following.”

Remove all the diff markings from `[meta.trans.other]`.

Renumber Table 57 (since it is no longer referring to a table in another document).

Change the description of `common_type` in `[meta.trans.other]` as follows (includes resolutions of [#235](#) and [#420](#)):


```

2. Note A: For the common_type trait applied to a parameter pack [...]
[...]
(2.3)      - Otherwise, if sizeof...(T) is two, let T1 and T2 denote the
              two types in the pack T, and let D1 and D2 be decay_t<T1> and
              decay_t<T2> respectively. Then
(2.3.1)    - If D1 and T1 denote the same type and D2 and T2 denote
              the same type, then
(2.3.1.1) - If COND_RES(T1, T2) is well-formed, then the
+ (2.3.1.1) - If std::common_type_t<T1, T2> is well-formed, then the
              member typedef type denotes
              decay_t<COND_RES(T1, T2)>.
+              std::common_type_t<T1, T2>.
(2.3.1.1.1) - If COMMON_REF(T1, T2) is well-formed, then the
              member typedef type denotes that type.
(2.3.1.1.2) - Otherwise, there shall be no member type
[...]
(2.4.2)    - Otherwise, there shall be no member type.
-?- Note B: Notwithstanding the provisions of (\cxxref{meta.type.synop}),
and pursuant to [...].

```

424: Constrain return types in IndirectInvocable

Both range-v3 and cmcstl2 constrain the return types of the required expressions of the IndirectInvocable concepts to share a common reference. The concepts in STL2 don't seem to do that. Was this just an oversight?

A polymorphic function F that has wildly unrelated return types when invoked with a value_type_t<I>& vs. a reference_t<I> should not be accepted by the STL algorithms because of the havoc it would cause.

Proposed Resolution

This resolution assumes the switch to variable-style concepts suggested by [P0651](#).

Change IndirectUnaryInvocable ([indirectcallable.indirectinvocable]) as follows:

```

template <class F, class I>
concept bool IndirectUnaryInvocable =
    Readable<I> &&
    CopyConstructible<F> &&
    Invocable<F&, value_type_t<I>&> &&
    Invocable<F&, reference_t<I>> &&
    Invocable<F&, iter_common_reference_t<I>>>;
+   Invocable<F&, iter_common_reference_t<I>> &&
+   CommonReference<
+       result_of_t<F&(value_type_t<I>&>),
+       result_of_t<F&(reference_t<I>&&>>>;

```

Change IndirectRegularUnaryInvocable ([indirectcallable.indirectinvocable]) as follows:

```

template <class F, class I>
concept bool IndirectRegularUnaryInvocable =
    Readable<I> &&
    CopyConstructible<F> &&
    RegularInvocable<F&, value_type_t<I>&> &&
    RegularInvocable<F&, reference_t<I>> &&
    RegularInvocable<F&, iter_common_reference_t<I>>>;
+   RegularInvocable<F&, iter_common_reference_t<I>> &&
+   CommonReference<
+       result_of_t<F&(value_type_t<I>&>),
+       result_of_t<F&(reference_t<I>&&>>>;

```

436: `common_iterator`'s destructor should not be specified in `[common.iter.op=]`

Resolved by the proposed resolution of [#250](#).